CAPSTONE PROJECT 06 | NEW (v4)

LegacyLift: Modernise a Legacy Order-Management App

Sakura Telecom Accessories, a telecom retailer in India and Japan whose order desk still runs on a 12-year-old Python script used by 40 store staff.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

1. The customer's problem

The order-management script mixes data, business rules and printing in one file, uses global state and a deprecated date API, has no tests, and depends on a library version with a known vulnerability. Every change breaks something. The company wants it modernised into a FastAPI + React application without changing what the business relies on.

2. Users

- Store staff who create and update orders
- Store managers who approve discounts
- The IT team who maintain the system
- Security and audit

3. User stories

- As store staff, I create an order and get the same totals, tax and discounts as the old system.
- As a store manager, discounts above 10% wait for my approval.
- As IT, I can change a pricing rule and the tests tell me if anything else broke.
- As security, I see no known vulnerable dependencies and no secrets in the code.
- As the product owner, I see a migration log that shows each step was verified.

4. Requirements

Must have

- Characterisation tests that pin the legacy behaviour before any change (totals, tax, discounts, status rules)
- Assessment: a module map, risks and a migration plan (rehost, replatform, refactor or rebuild) per module (Day 23)
- FastAPI service with Pydantic models and SQLAlchemy persistence, matching legacy outputs on the characterisation suite (Days 5 to 7)
- React order screen with loading, error and empty states (Days 10, 11)
- Migration done in small, verified pull requests with OpenAI Codex; every PR keeps the suite green
- Vulnerability report before and after: bandit, pip-audit, npm audit (Days 7, 20, 23)
- An AI feature: an order-notes summariser or a migration assistant that explains legacy functions, with token and cost logging (Days 12, 25)

Should have

- Strangler pattern: old and new run side by side behind one interface, with a comparison report
- Discount approval with an audit log
- CI pipeline with security gates and a quality gate

Could have (stretch)

- Data migration script with reconciliation counts
- Japanese and English invoice output
- A Codex skill or AGENTS.md section that documents the team's migration recipe

5. Data

Starter notebook writes the legacy module (legacy_orders.py), 300 sample orders and a characterisation test file that passes against the legacy code.

6. Suggested AI approach

Assess, plan, migrate with a coding agent, verify with tests (Day 23); brownfield modernisation with Codex, tests first (Day 30); security gates (Day 20); token and cost logging (Days 12, 25).

7. Suggested architecture

```
React UI -> FastAPI back end -> database (SQL / MongoDB)
              |-> AI service (retrieval, model, tools, guardrails)
              |-> logs, tokens and cost -> dashboard / weekly report
CI/CD: tests + security scans + quality gate -> Docker images
       -> Azure (Container Apps or App Service)
```

8. v4 baseline for this project

Modernisation step: The whole project is the modernisation. Start with legacy_orders.compute_total() and its characterisation tests, then migrate one function per pull request.

- **AGENTS.md in the team repo:** build, test and style commands, folders not to touch, and "never put secrets in prompts" (Day 3, template in shared/templates/AGENTS_learner_template.md).
- **Security checklist** mapped to the OWASP Top 10 for Agentic Applications (shared/templates/OWASP_Agentic_Checklist.md; Days 23, 30).
- **Vulnerability report** from bandit, pip-audit, npm audit and your red-team suite (shared/templates/Vulnerability_Report_Template.md; Days 7, 20, 30).
- **Token and cost report:** tokens and estimated cost per request and per 1,000 requests, before and after one optimisation, with placeholder prices labelled (Days 12, 25, 33).

9. How to do it: day by day

Day 36 (Tue 15 Dec 2026): Kick-off and design

- Quiz on Day 35
- Teams of 3 to 4 formed; project chosen

- Problem statement, users, success metrics, assumptions log
- Architecture diagram and data model
- Repository, board and CI skeleton created, with AGENTS.md in the repo root (NEW v4)
- Pick your modernisation step and write its characterisation tests first (NEW v4)

Day 37 (Wed 16 Dec 2026): Core build

- Stand-up (15 min)
- Data loaded; back end API with main endpoints
- AI core working end to end (retrieval, recommendation, agent or pipeline)
- Golden set draft of 20 items
- Token and cost logging on every model call (NEW v4)

Day 38 (Thu 17 Dec 2026): Complete features and quality

- Stand-up
- Front end connected to the API
- Guardrails and error handling
- Evaluation run and first error analysis
- Tests in CI; quality gate added
- Modernisation step merged through a reviewed pull request (NEW v4)

Day 39 (Fri 18 Dec 2026): Harden, deploy, observe

- Stand-up
- Docker images; deploy to Azure or a documented local deployment
- Structured logs and a monitoring view
- Red-team and slice checks; fix top issues
- Security checklist (OWASP Top 10 for Agentic Applications) and vulnerability report completed (NEW v4)
- Token and cost report written (NEW v4)
- Demo rehearsal with peer feedback
- Code freeze at 17:00 IST

Day 40 (Mon 21 Dec 2026): Demo day

- Deliverables due at 09:00 IST
- Final rehearsal (morning)
- Demo: 10 minutes + 5 minutes Q&A per team
- Evaluator scoring with the rubric
- Individual reflection and peer feedback
- Closing and certificates

10. Project-specific acceptance checks

Evaluators run these during or after the demo. They count towards Core functionality and AI quality.

- Characterisation suite passes against both legacy and new code
- No known vulnerable dependency remains (pip-audit clean)
- Discounts above 10% are never applied without approval

- Every migration step has its own reviewed pull request

11. Golden set: starter items

Extend these to at least 20 items (30 for PolicyPal) with expected answers, sources and tags.

- Order of 3 x 1,200 INR cases with 18% GST must total 4,248.00 INR in both systems
- A 15% discount must go to the approval queue, not be applied
- An order with an unknown product id must be rejected with a clear error

12. How you will be evaluated

Criterion	Points	What evaluators look for
Problem understanding and scoping	10	Clear problem statement, users, success metrics, assumptions log, agreed MVP scope (FDE Mindset Days 1, 14, 19).
Architecture and design	10	Sensible architecture diagram, layered code, right tool choices with reasons, data model (Days 4, 6, 18, 34).
Core functionality	20	All must-have features work end to end in the demo; project checklist items delivered.
AI and data quality	15	Golden set of at least 20 items, measured metrics (retrieval, faithfulness or recommendation quality), error analysis (Days 22, 31, 32).
Engineering quality	10	Readable code, tests (API, pipeline, prompts), Git history with pull requests, CI pipeline green (Days 15, 17, 20). NEW (v4): AGENTS.md in the repo; modernisation step done tests first.
Security, guardrails and responsible AI	10	Input and output guardrails, secrets handling, least privilege, a red-team check, fairness or language slice check (Days 7, 30). NEW (v4): OWASP Agentic checklist and vulnerability report.
Deployment and operations	10	Containerised, deployable (Azure or documented), structured logs, a dashboard or report, one drift or quality check (Days 21, 33 to 35). NEW (v4): token and cost report.
Demo and communication	10	Answer-first readout, live demo, honest limits, confident Q&A, clear handover README (FDE Mindset Days 24, 29).
Teamwork and process	5	Daily stand-ups, decision log, fair contribution, kind and specific feedback (FDE Mindset Days 14, 24, 29).

13. Common pitfalls

- Rewriting everything at once without tests
- Changing behaviour while migrating (fix bugs in a separate, labelled PR)
- Accepting Codex diffs you cannot explain

14. Getting started

- Open Starter_Notebooks/Project06_LegacyLift_Starter.ipynb in Colab or VS Code to generate the data and see a working baseline.

- Use App_Build_Prompts/Project06_LegacyLift_Prompts.md with OpenAI Codex to scaffold the project. Review every diff.
- Write your problem statement and assumptions log before writing code.

Ganbatte (頑張って): do your best